THE UNIVERSITY OF MELBOURNE

COMP30023 – Computer Systems

CPU Scheduling

Maria Rodriguez Read

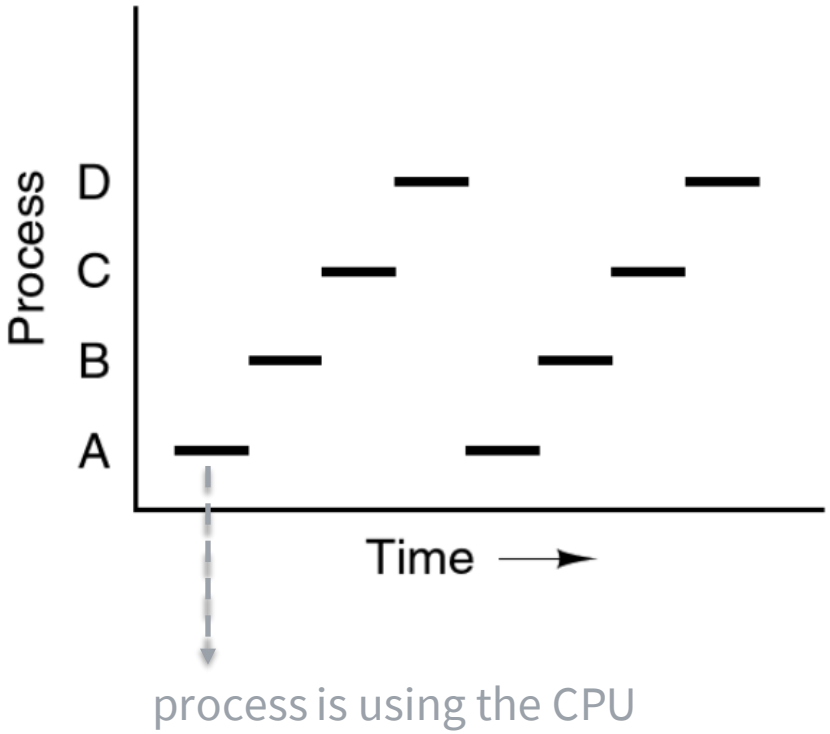
Semester 1, 2025

© University of Melbourne

Need for CPU Scheduling

- Only one process can run at a time in a (single-core) CPU => multiprogramming requires **switching** the CPU among processes
- A **scheduling algorithm** determines which process/thread gets to run next

Based on TB Figure 2-1



Process vs. Thread Scheduling

- The scheduling unit can be a (single-threaded) process or a thread
- This lecture:
  - Focuses on processes as the scheduling unit
  - But the scheduling policies we cover are task-agnostic and can be used in both cases
- For awareness, a few thread-specific approaches are highlighted in the last slide (non-examinable)

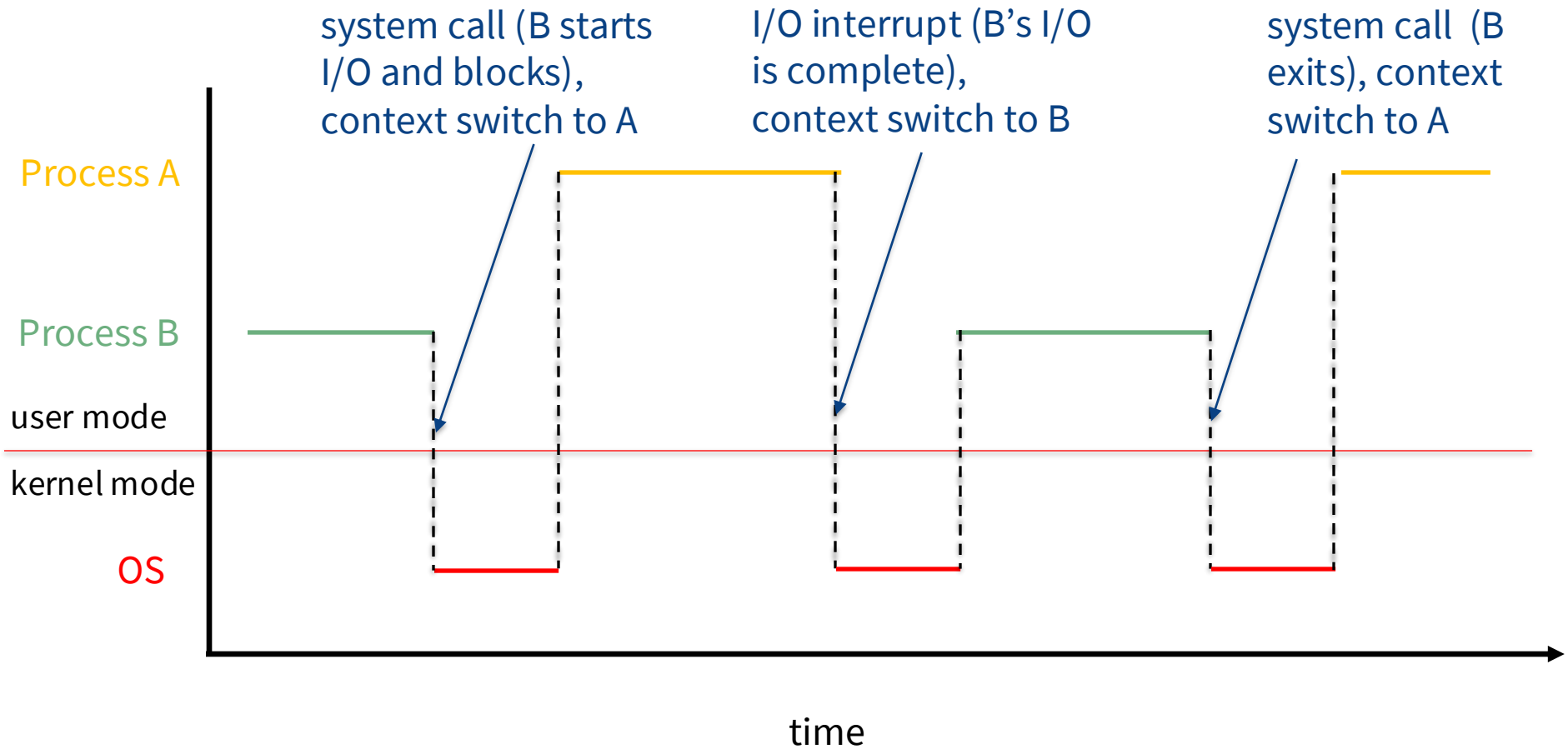
© University of Melbourne

Context Switch

- Changing execution contexts, i.e., transitioning from executing one process/thread to another
- A **thread context switch** (switch between threads of the same process) includes
  - Saving the execution context of the current thread to the TCB
  - Loading the execution context of another thread from the TCB to the CPU
- A **process context switch** (switch between threads of different processes) includes
  - Saving/loading execution contexts (as in a thread context switch), plus
  - Loading state related to memory management from the PCB (e.g., pointer to page table), plus
  - Flushing the Translation Lookaside Buffer (in a paged system with virtual memory)

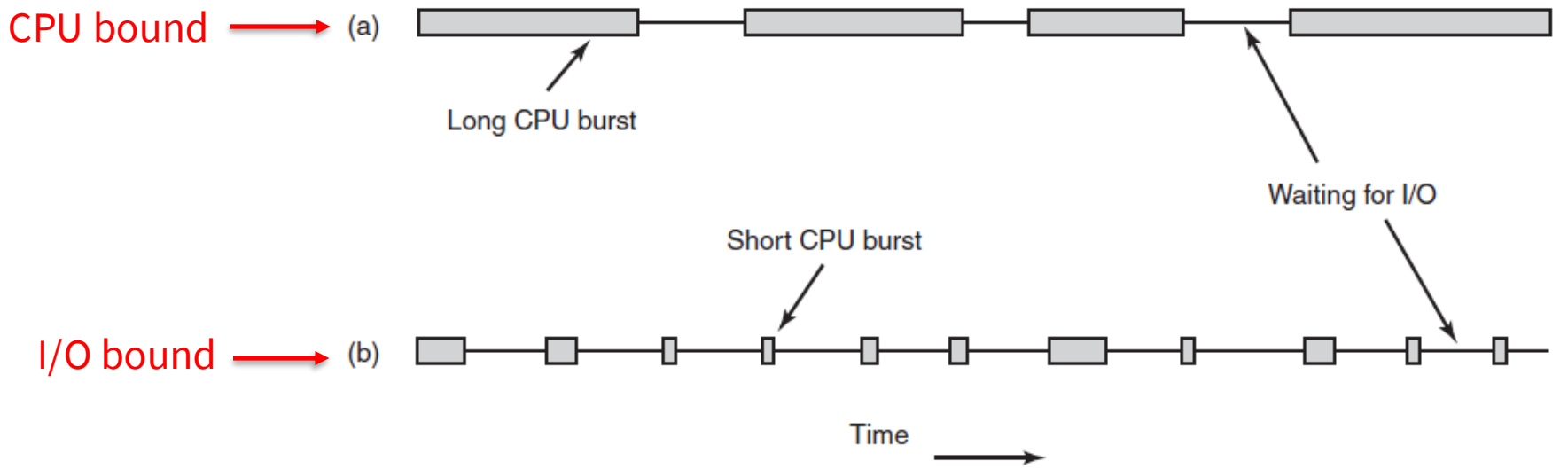
more on this in Week 4

Context Switch – Sample Scenario



Process Behavior

- Most processes alternate bursts of computing with I/O requests (e.g., disk or network)
- **CPU bound processes**: Have long CPU bursts. Spend most of their time computing.
- **I/O bound process**: Have short CPU bursts. Spend most of their time waiting for I/O.



TB 2-39

CPU Scheduling

- Pick a process to *run* from a set of processes that are *ready* to execute
- Recall that *blocked* processes cannot *run*, e.g.,
  - If a process is *running* and initiates an I/O operation, it *blocks*, and the scheduler finds another process to *run*

Processes that are *ready* to run

P_6	P_5	P_4	P_3	P_2	P_1

© University of Melbourne

Preemptive vs. Non-preemptive Scheduling

- **Non-preemptive scheduling**
  - Pick a process to run and let it run until it blocks or voluntarily releases the CPU
  - Context switches are performed only when necessary (i.e., when a process cannot use the CPU)
- **Preemptive scheduling**
  - Pick a process and:
    - Let it run for a maximum of some fixed time (a quantum)
    - If it is still running at the end of the quantum, it is suspended, and the scheduler picks another process to run
  - Clock interrupts are needed to give control of the CPU back to the scheduler
  - Typically results in more context switches

© University of Melbourne

Scheduling Environments

- **Batch systems**
  - A set of (potentially periodic) jobs that need to run
  - Jobs don't require interaction with users
  - Non-preemptive scheduling and preemptive scheduling with a long quantum are often a good choice for these environments
- **Interactive systems**
  - Interactive users expect fast response times/responsiveness from the system
  - Preemption is essential

© University of Melbourne

Scheduling Goals

- All systems
  - **Fairness**
    - Processes get a fair share of the CPU
    - Comparable processes should get comparable service
  - **Efficient use of resources**
    - For example:
      - Allowing I/O bound jobs to use the CPU when they need it so that they can keep the I/O devices busy
      - Reducing overhead by minimising context switches

© University of Melbourne

Scheduling Goals

- Batch systems
  - Maximise **throughput**
    - Number of processes that are completed per time unit
  - Minimise **turnaround time**
    - The interval from the time of submission of a process to the time of completion
- Interactive systems
  - Minimise **response time**
    - The time between issuing a command and getting a result

© University of Melbourne

Non-preemptive Scheduling Algorithms

- First-come First-served
- Shortest Job First

© University of Melbourne

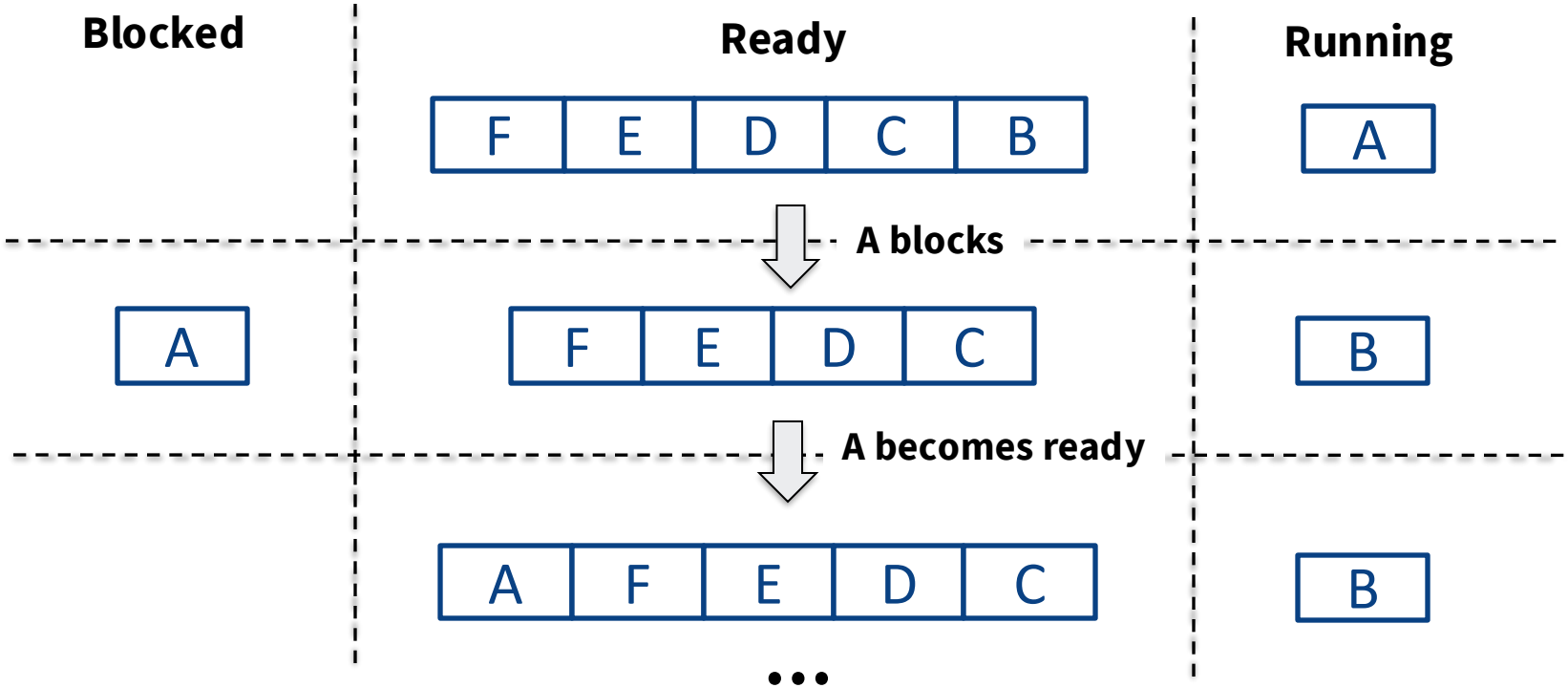
First-come First-served (FCFS)

- Run processes in the order in which they become ready to execute
- When the currently running process stops executing (e.g., blocks), the process that has been in the ready queue the longest is selected for running
- FIFO queue
  - Process at the head of the queue gets to run
  - Append newly ready processes to the tail of the queue

© University of Melbourne

FCFS Example

FIFO queue *of ready* processes



FCFS Analysis

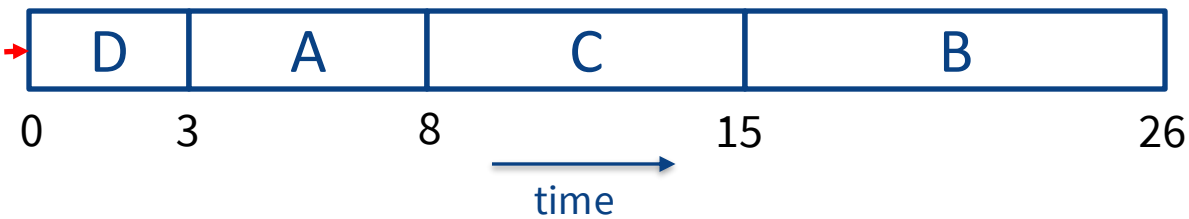
- Simple, easy to implement, low context switching overhead
- Convoy effect
  - Short processes (with short CPU bursts) waiting behind a long CPU-bound process
  - I/O-bound processes
    - Might spend a lot of time queuing up
    - Reduced resource utilisation (e.g., I/O devices are sitting idle while the I/O bound processes wait for the CPU)
  - Longer turnaround times
- Starvation is possible if a ready process is entirely CPU-bound (no I/O operations)

© University of Melbourne

Shortest Job First (SJF)

Schedule the job that has the least amount of work to do (shortest CPU burst) until its next I/O request or completion

execution using SJF



Process	Burst Time
A	5
B	11
C	7
D	3

Assume all processes became *ready* at time 0

Turnaround Time (time of completion – time of arrival):

$T_D = 3 - 0 = 3$ $T_A = 8 - 0 = 8$ $T_C = 15 - 0 = 15$ $T_B = 26 - 0 = 26$

$\mathbf{T_{AVG}} = (3 + 8 + 15 + 26)/4 = 52/4 = 13$

© University of Melbourne

SJF Analysis

- Requires estimating the length of CPU bursts
- Optimal average turnaround time (for a given set of *jobs* that are available simultaneously)
  - In an OS context, job = next CPU burst of a process
- Processes with long CPU bursts can be starved (if processes with shorter bursts continuously arrive/become ready to run)

© University of Melbourne

Preemptive Scheduling Algorithms

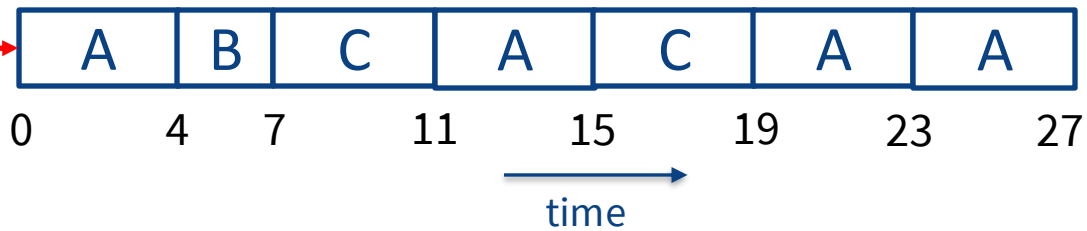
- Round Robin
- Priority Scheduling
- Multi-level Feedback Queue

© University of Melbourne

Round Robin (RR)

- Run a process for a quantum, then switch to the next job in the *ready* queue. Repeat until all processes are finished
- If a process blocks/completes before its quantum, pick the next process

execution using RR (quantum = 4)



Process	Ready Time	Burst Time
A	0	16
B	2	3
C	3	8

Turnaround Time:

$T_A = 27 - 0 = 27$

$T_B = 7 - 2 = 5$

$T_c = 19 - 3 = 16$

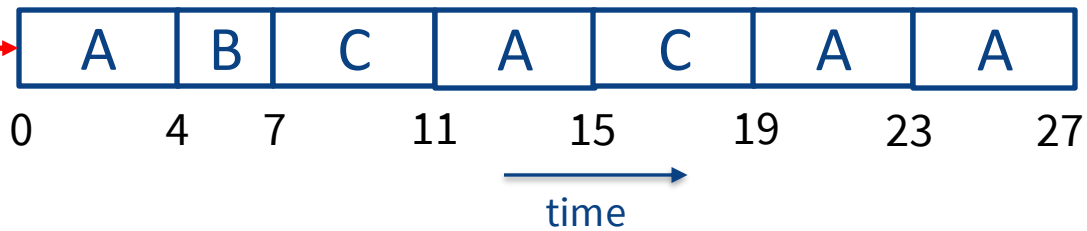
$\mathbf{T_{AVG}} = (27 + 5 + 16)/3 = 16$

© University of Melbourne

Round Robin (RR)

- To estimate the response time, let's assume that a (CPU burst of a) process produces a response as soon as it runs:
  - Response time = time of first run – arrival time (to the ready queue)

execution using RR (quantum = 4)



Process	Ready Time	Burst Time
A	0	16
B	2	3
C	3	8

Response Time:

$R_A = 0 - 0 = 0$

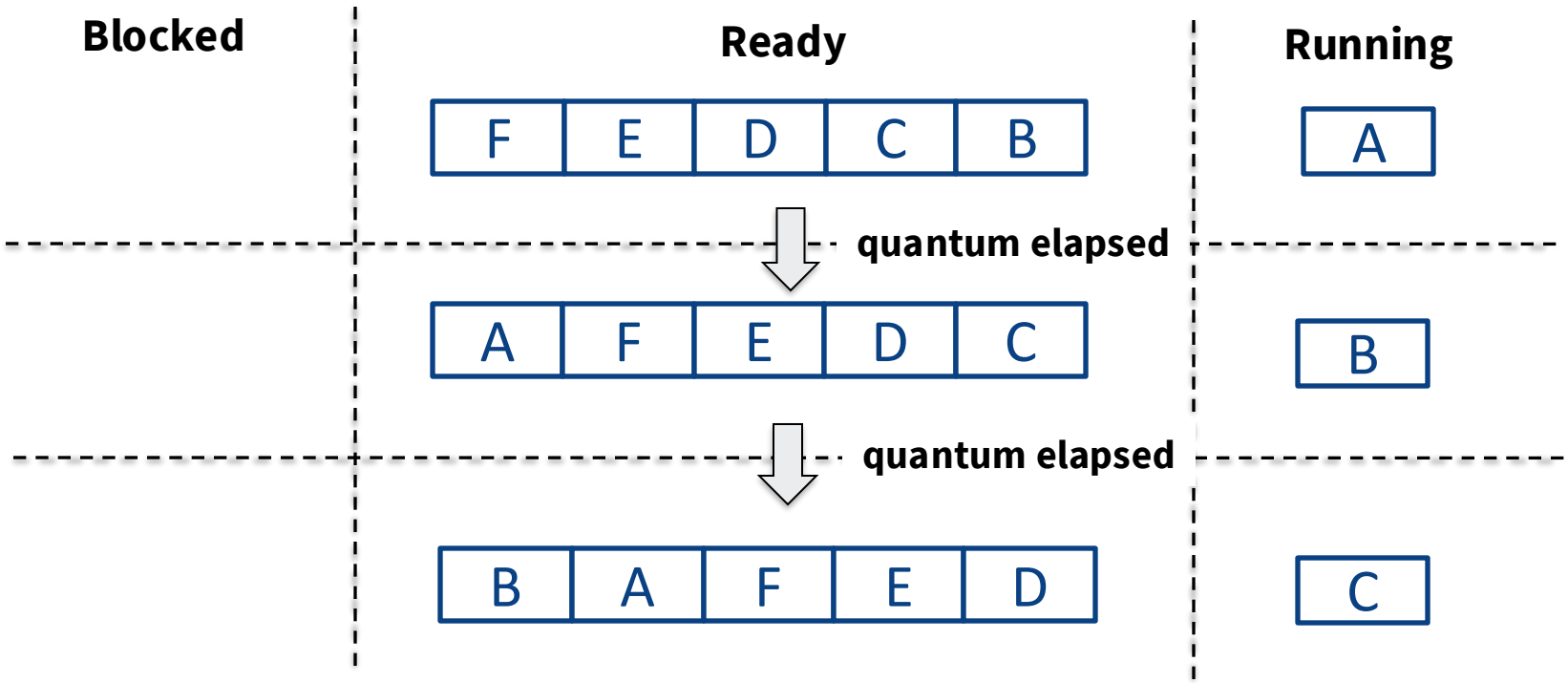
$R_B = 4 - 2 = 2$

$R_c = 7 - 3 = 4$

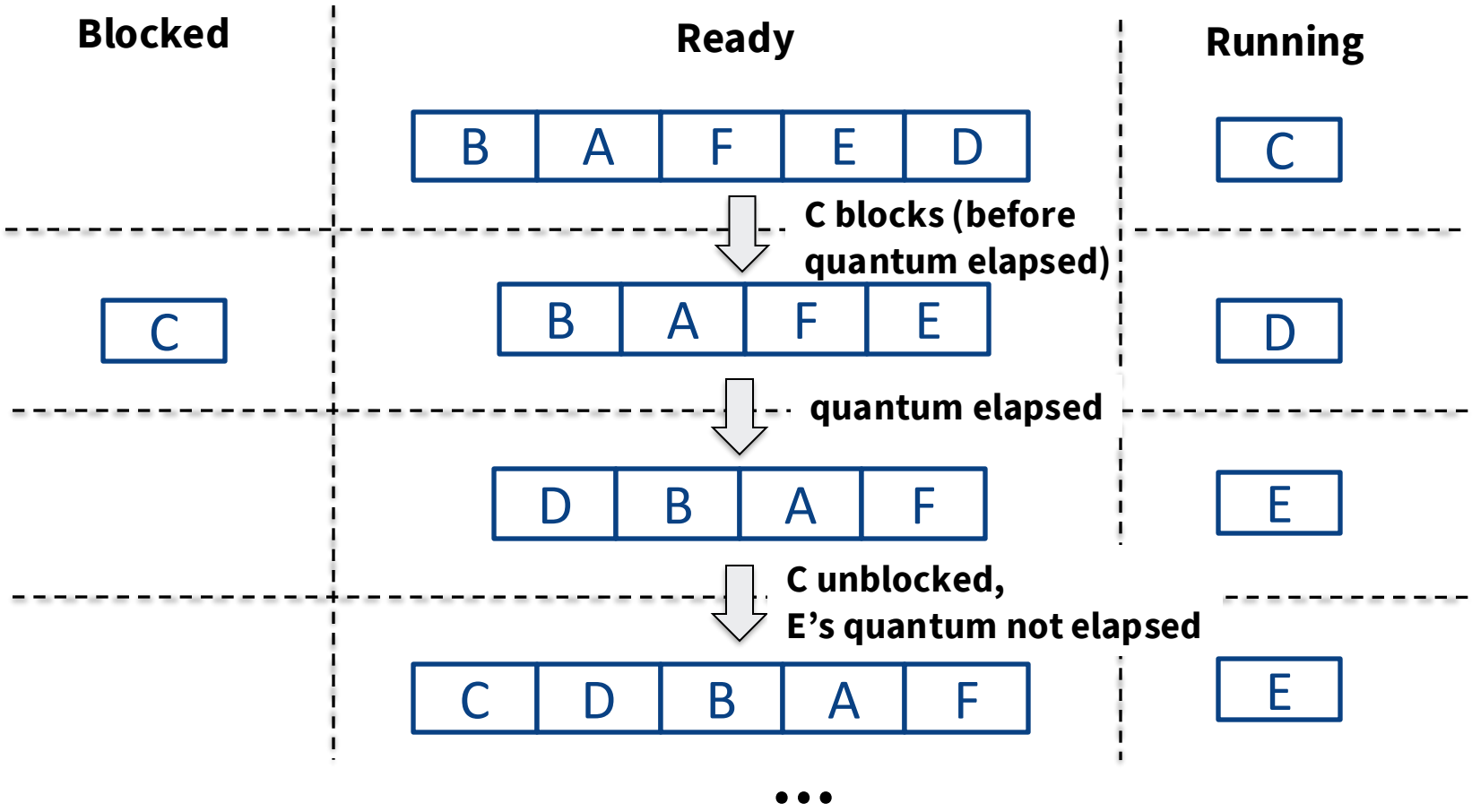
$\mathbf{R_{AVG}} = (0 + 2 + 4)/3 = 2$

RR Example

queue *of ready* processes



RR Example (contd.)



© University of Melbourne

Length of Quantum in RR

- Long(er)
  - Too long, RR morphs into FCFS
  - Less context switching overheads
  - Less responsive
- Short(er)
  - More context switching overhead
  - More responsive
- The length of the quantum presents a trade-off:
  - Making it long enough to amortise the cost of context switching without making it so long that the system is no longer responsive

© University of Melbourne

RR Analysis

- Simple, easy to implement
- With a reasonable quantum:
  - Not good for turnaround time
  - Good for response time
- Fair?
  - Tends to favour CPU-bound processes over I/O-bound ones
- Overhead of context switching depends on length of quantum
- No starvation

© University of Melbourne

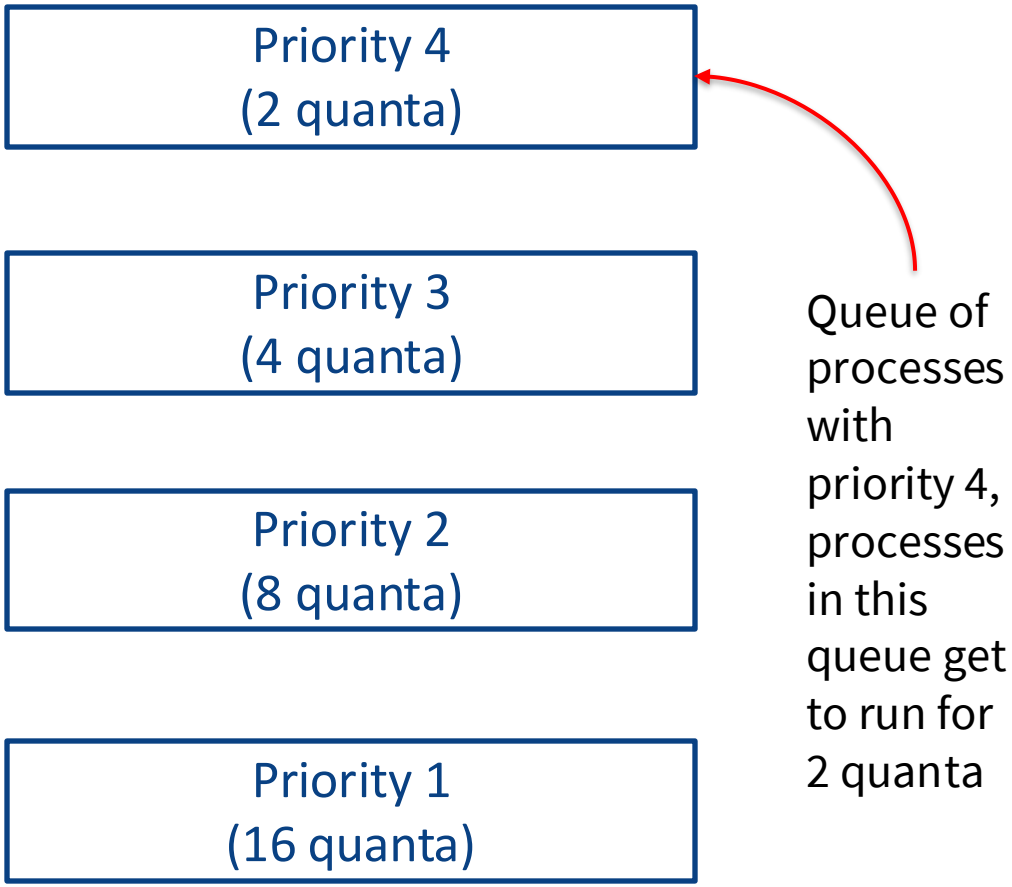
Priority Scheduling

- Assign a priority to each job, allocate the CPU to the highest priority process that is *ready* to run
- Priorities can be assigned:
  - Statically, at process creation.
  - Dynamically, adjusted throughout the execution of a process
- To avoid starvation:
  - Change the priority of the running process with time, e.g., decrease priority after each clock interrupt (clock tick)
  - Preempt process if its priority dropped below that of the next highest process

© University of Melbourne

Multi-Level Feedback Queue (MLFQ)

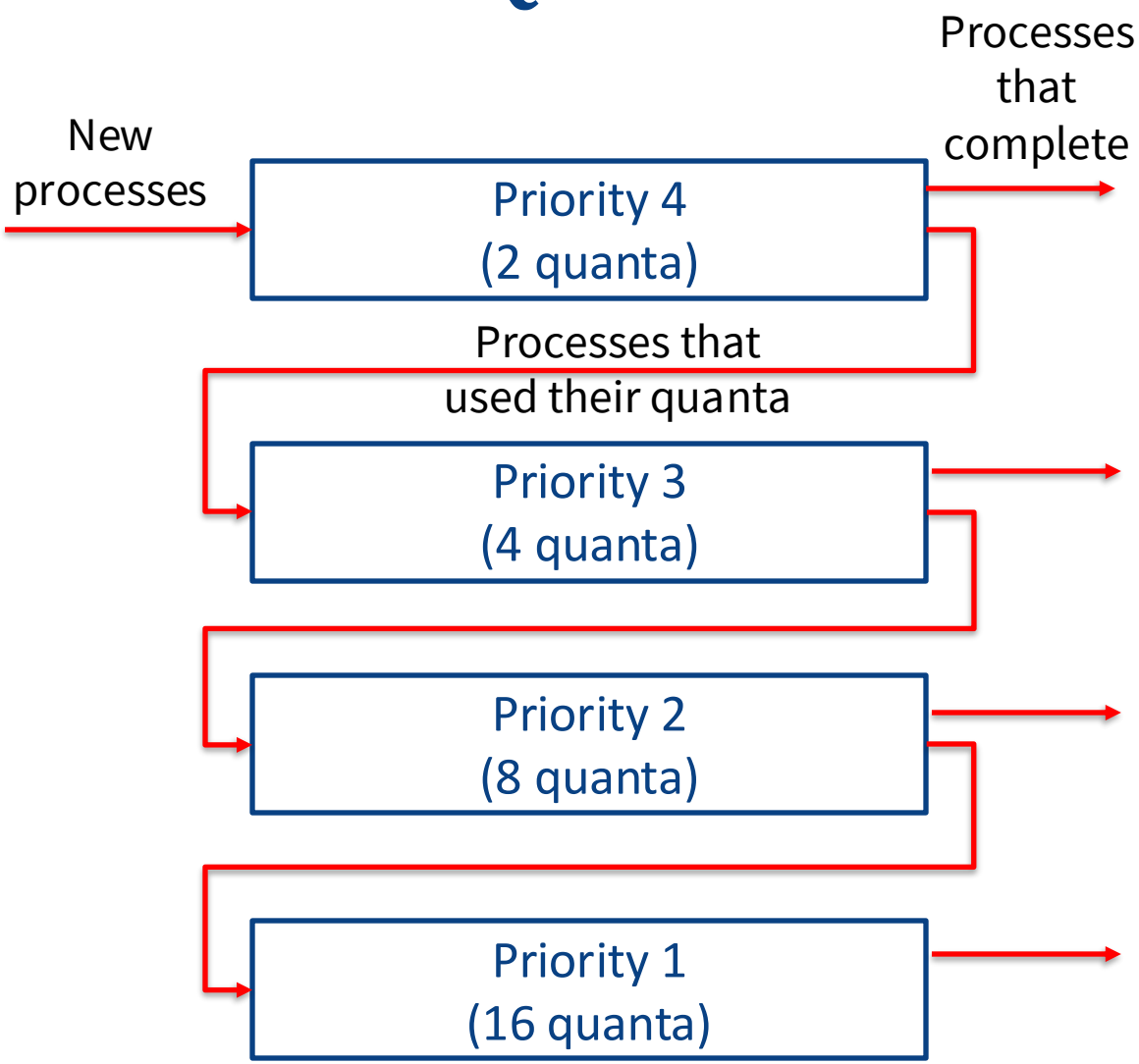
- Multiple queues, each with a different priority
- The quanta processes are allowed to run for increases as the priority of the queue decreases



© University of Melbourne

Multi-Level Feedback Queue (MLFQ)

- Processes start at the highest priority queue
  - Used their quanta? Move to a lower priority queue
  - Haven't used all their quanta? Stay in the current priority queue
- Processes in the queue with the highest priority get to run
- Algorithm used to schedule processes within the same queue is a parameter to the algorithm – e.g., RR



MLFQ Analysis

- I/O-bound processes are given priority over CPU-bound ones
  - Good performance for short-running interactive jobs
  - Makes progress for long-running CPU-intensive workloads
- CPU-bound processes get a large quantum every once in a while, rather than small one frequently
- No a priori knowledge of processes needed. Instead, it observes their behavior and prioritizes them accordingly
- Parameters include
  - Number of queues
  - Scheduling algorithm per queue
  - Quantum for each queue

© University of Melbourne

Thread-focused Policies (NOT EXAMINABLE)

- Treat as a generic task, i.e., work that the processor needs to do
  - E.g., use any of the scheduling algorithms we learnt
- Gang scheduling (in multiprocessors only)
  - All threads of a process run at the same time across multiple cores
- Processor affinity (in multiprocessors only)
  - Assign threads to specific processors, improve cache hit ratio
- Schedule processes, and within each process, schedule threads
  - Reduce overhead of context switching, improve cache hit ratio

© University of Melbourne